arXiv:2412.01253v5 [cs.CL] 22 Jan 2025

Yi-Lightning Technical Report

01.AI

Abstract

This technical report presents **Yi-Lightning**, our latest flagship large language model (LLM). It achieves exceptional performance, ranking **6th** overall on Chatbot Arena, with particularly strong results (2nd to 4th place) in specialized categories including Chinese, Math, Coding, and Hard Prompts. Yi-Lightning leverages an enhanced Mixture-of-Experts (MoE) architecture, featuring advanced expert segmentation and routing mechanisms coupled with optimized KV-caching techniques. Our development process encompasses comprehensive pre-training, supervised fine-tuning (SFT), and reinforcement learning from human feedback (RLHF), where we devise deliberate strategies for multi-stage training, synthetic data construction, and reward modeling. Furthermore, we implement RAISE (Responsible AI Safety Engine), a four-component framework to address safety issues across pre-training, post-training, and serving phases. Empowered by our scalable super-computing infrastructure, all these innovations substantially reduce training, deployment and inference costs while maintaining high-performance standards. With further evaluations on public academic benchmarks, Yi-Lightning demonstrates competitive performance against top-tier LLMs, while we observe a notable disparity between traditional, static benchmark results and real-world, dynamic human preferences. This observation prompts a critical reassessment of conventional benchmarks' utility in guiding the development of more intelligent and powerful AI systems for practical applications. Yi-Lightning is now available through our developer platform at `https://platform.lingyiwanwu.com`.

Contents

1 Introduction **3**

2 Model Architecture **4**

2.1 Fine-grained Expert Segmentation 4

2.2 Expert Routing Strategy 4

2.3 KV Cache Reduction 5

3 Pre-training **5**

3.1 Data 6

3.2 Training Strategy 6

3.3 Long Context Extension 6

4 Post-training **7**

4.1 Supervised Fine-tuning 7

4.1.1 Data and Training Strategy 7

4.1.2 Optimized Implementations 7

4.2 Reinforcement Learning from Human Feedback 7

4.2.1 Reward Modeling 8

4.2.2 Preference Data 8

4.2.3 Direct Preference Optimization 8

5 Infrastructure **9**

5.1 Parallelism Optimization 9

5.2 Inference Optimization 9

5.3 Goodput Optimization 10

6 Safety **10**

7 Evaluation **11**

A Author List and Contributions **13**

1 Introduction

Large language models (LLMs) have revealed fascinating prospects toward artificial general intelligence (AGI), attracting an enduring enthusiasm and interest of the community [Achiam et al., 2023, Gemini Team et al., 2023, Dubey et al., 2024, Yang et al., 2024, 01.AI, 2024]. With our mission in mind to empower the community with advanced AI technology and exceptional AI service experiences, we release this technical report and introduce our new-generation flagship model, **Yi-Lightning**. As of its first appearance on October 16, 2024, Yi-Lightning achieved a remarkable overall ranking of **6th** place on the Chatbot Arena leaderboard [Zheng et al., 2023b] (as shown in Figure 1), a leading LLM benchmark based on real-world human judgment and comparison. In specialized categories such as Chinese, Math, Coding, and Hard Prompts, it also ranks among the top performers (ranging from 2nd to 4th place), demonstrating a comprehensive high-performance standard in practical scenarios.

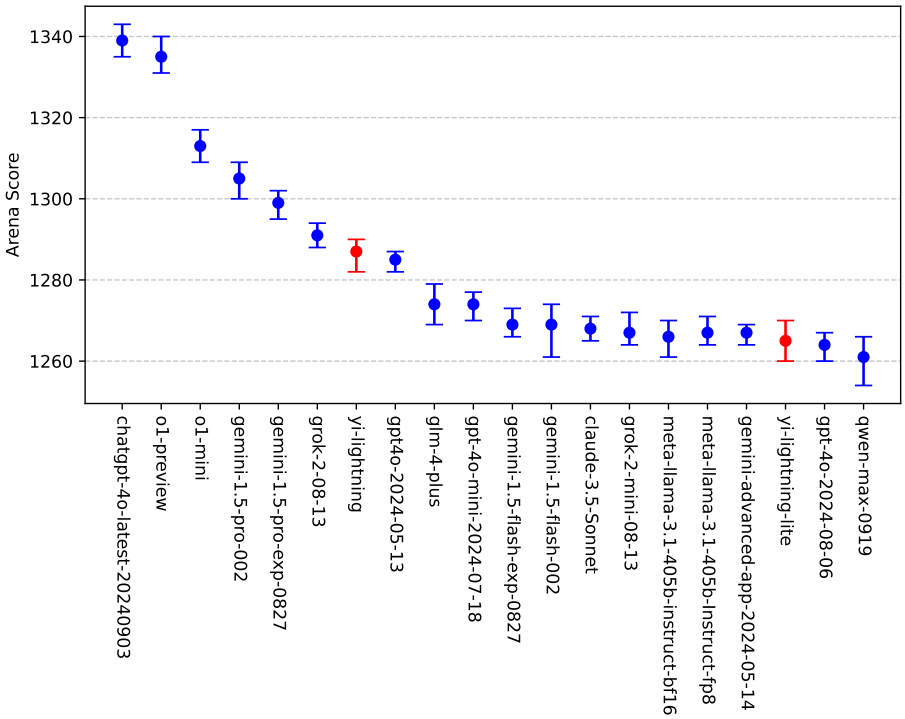

Figure 1: Snapshot of the Chatbot Arena leaderboard on October 16, 2024, with Yi-Lightning's initial appearance. According to the official Chatbot Arena leaderboard, Yi-Lightning ranked **6th** overall, tied with Grok-2-08-13.

We attribute Yi-Lightning's excellent performance to our innovations in model architecture (§ 2), training strategies, data engineering (§ 3 and § 4), and infrastructure (§ 5). These efforts work closely together, consequently enabling Yi-Lightning's efficient training, deployment, inference, and its high practical efficacy:

- In terms of **model architecture**, Yi-Lightning is based on an improved mixture of experts architecture. It employs fine-grained expert segmentation, complemented by a balanced expert routing strategy and cross-layer KV cache sharing design, achieving more efficient training and inference.
- Regarding **training strategies**, Yi-Lightning extensively utilizes specialized multi-stage and strategic training recipes in pre-training, supervised fine-tuning, reward modeling, and human preference alignment optimization, achieving more efficient performance optimization.
- In **data engineering**, building upon general domain foundations, we design data synthesis approaches for difficult and complex tasks (such as mathematics and coding), significantly enhancing Yi-Lightning's problem-solving capabilities.

- For **infrastructure**, we implement substantial optimizations in parallelism strategies, node management and scheduling, storage, and network communication, greatly improving goodput performance.
- Additionally, we implemented RAISE, a four-component framework to address the **safety** concerns across Yi-Lightning's entire lifecycle from development to deployment.

Finally, we report Yi-Lightning's evaluation results on public academic benchmarks. While Yi-Lightning still performs competitively against other top-tier LLMs, we observe a notable disparity between the evaluation results on these academic benchmarks and real-world human judgments on Chatbot Arena, which probably results from our focus on optimizing toward practical experiences rather than overly paying attention to the benchmark scores. This observation stimulates us to reconsider the role of the current academic benchmarks in guiding more intelligent and powerful AI systems and to design alternative approaches to evaluating model performance in practical scenarios.

2 Model Architecture

Similar to recent large language models [Dai et al., 2024, Yang et al., 2024, Jiang et al., 2024a, Abdin et al., 2024], Yi-Lightning is fundamentally built upon the Mixture-of-Experts (MoE) architecture. Beyond this, we introduce several architectural innovations in Yi-Lightning, including fine-grained expert segmentation methodology, advanced routing strategies, and optimized key-value cache reduction techniques.

2.1 Fine-grained Expert Segmentation

Recent research has revealed that as dense models grow in size, their activation patterns become increasingly sparse [Zhang et al., 2024b,a]. This sparsity indicates that parameters are not uniformly utilized during inference, leading to computational inefficiencies. The Mixture-of-Experts (MoE) architecture addresses this challenge by selectively routing tokens to activate only specific neural subsets. However, even with MoE models, the issue of sparse activations persists within individual experts, suggesting a fundamental challenge in parameter utilization efficiency.

Drawing inspiration from [Dai et al., 2024], we adopt a fine-grained expert segmentation strategy. This approach involves partitioning each expert's Feed-Forward Network (FFN) into smaller functional units, simultaneously reducing intermediate hidden dimensions while increasing the number of experts activated per token. This fine-grained segmentation facilitates more nuanced knowledge decomposition, enhances expert activation combinations, and improves overall parameter utilization efficiency. In practice, we observed that excessive expert segmentation substantially impacted training throughput. Consequently, we opted for a balanced approach, implementing segmentation only to the extent necessary to maintain optimal training efficiency rather than pursuing maximum segmentation for performance gains.

2.2 Expert Routing Strategy

Expert routing strategy plays a crucial role in optimizing training efficiency and model quality. Following Switch-Transformer (ST) [Fedus et al., 2022], we initially implemented a load balancing mechanism with an auxiliary loss function for N experts and a batch $\mathcal{B}$ containing $|\mathcal{B}|$ tokens:

$$\mathcal{L}_{\text{ST}} = \alpha_{\text{ST}} \cdot N \cdot \sum_{i=1}^{N} f_i \cdot P_i.$$

Here, f_i represents the fraction of tokens x routed to each expert, and P_i denotes the average routing probability allocated to expert i:

$$f_i = \frac{1}{|\mathcal{B}|} \sum_{x \in \mathcal{B}} \mathbb{1} \left[\left(\arg\max_{1 \le j \le N} p_j(x) \right) = i \right], \quad P_i = \frac{1}{|\mathcal{B}|} \sum_{x \in \mathcal{B}} p_i(x),$$

where $p_i(x)$ represents the probability of token x being assigned to expert i. Given that $\sum_{i=1}^{N} f_i = 1, \sum_{i=1}^{N} P_i = 1$, it can be easily shown via the Lagrange multiplier method that the loss function

reaches its minimum when tokens are evenly distributed, with both f_i and P_i approaching $1/N$ for all experts. Therefore, this mechanism can effectively encourage uniform routing and balanced utilization across experts during training.

However, we observed that while this load balancing effectively prevents expert collapse and enhances training efficiency, the per-expert constraints prove overly restrictive even with carefully tuned α_{ST}. We thus propose to relax the constraints from individual experts to Expert Parallel (EP) groups (see § 5.1) by introducing the **EP load balancing** mechanism, which optimizes the load balance within each EP group:

$$\mathcal{L}_{\text{EP}} = \alpha_{\text{EP}} \cdot N^g \cdot \sum_{i=1}^{N^g} f_i^g \cdot P_i^g \tag{1}$$

where N^g denotes the number of experts in the EP group, with group-specific calculations:

$$f_i^g = \frac{1}{|\mathcal{B}^g|} \sum_{x \in \mathcal{B}^g} \mathbb{1}\left[\left(\underset{1 \le j \le N}{\arg\max}\, p_j(x)\right) = i\right], \quad P_i^g = \frac{1}{|\mathcal{B}^g|} \sum_{x \in \mathcal{B}^g} p_i(x),$$

where $\mathcal{B}^g$ denotes the batch tokens allocated to the specific EP group.

Nevertheless, a significant limitation of the above load balancing approaches still remains, as they are unable to address the *token dispatching imbalance* during All-to-All communication in expert parallelism. This imbalance results in fluctuating computation and communication intensities across EP groups, reducing training efficiency. The issue is further compounded by expert segmentation as it increases tokens that need dispatching. To address this challenge and enable finer-grained load balancing control, we introduce **partitioned EP load balancing (PEP)**, which splits experts within each EP group into smaller partitions. This approach ensures balanced token distribution across partitions, gradually optimizing communication and computation loads. For a partition (within a group of N^g experts) containing N^p local experts, the loss is computed as:

$$\mathcal{L}_{\text{PEP}} = \alpha_{\text{PEP}} \cdot N^p \cdot \sum_{i=1}^{N^p} f_i^p \cdot P_i^p, \tag{2}$$

where f_i^p and P_i^p represent the token fraction and the average routing probability for expert i in this group partition, respectively:

$$f_i^p = \frac{1}{|\mathcal{B}^p|} \sum_{x \in \mathcal{B}^p} \mathbb{1}\left[\left(\underset{1 \le j \le N}{\arg\max}\, p_j(x)\right) = i\right], \quad P_i^p = \frac{1}{|\mathcal{B}^p|} \sum_{x \in \mathcal{B}^p} p_i(x).$$

To maintain effective load balancing, we optimize $\mathcal{L}_{\text{PEP}}$ in conjunction with $\mathcal{L}_{\text{ST}}$ and $\mathcal{L}_{\text{EP}}$. In practice, we carefully tuned and set α_{PEP}, α_{EP}, and α_{ST} to 10^{-3}, 10^{-4}, and 10^{-6}, respectively.

2.3 KV Cache Reduction

To enhance long-context processing while substantially reducing inference costs, we introduce two key architectural innovations. First, we observed that while most attention heads primarily focus on local context, only a small subset specializes in global information processing. Motivated by this, we implement **hybrid attention blocks** that combine three sliding window attention [Jiang et al., 2023] layers with one full attention layer, effectively capturing both local patterns and global dependencies. Second, we optimize memory utilization by **cross-layer KV cache reuse**, which shares key-value (KV) cache states between consecutive full attention layers and reduces memory requirements by half for full attention components. These innovations collectively achieve up to 82.8% memory reduction while maintaining model performance on long sequences.

3 Pre-training

We then describe the pre-training methodology of Yi-Lightning. While building upon the experience of training our previous Yi models [01.AI, 2024], we specifically optimize data processing and composition, closely integrating these improvements with pre-training progress to design an advanced multi-stage training approach.

3.1 Data

Our pre-training corpus comprises multilingual web documents (crawled through early 2024), books, academic papers, codebases, and question-answer pairs. Building upon the data processing pipeline from [01.AI, 2024], we strengthen our filtering mechanisms for unsafe content and personally identifiable information (PII), and particularly implement several key improvements detailed below.

Tokenization We employ byte-pair encoding (BPE) for text tokenization [Shibata et al., 1999] with the SentencePiece implementation [Kudo and Richardson, 2018]. In contrast to our previous Yi models [01.AI, 2024], we expand the vocabulary size to 100,352 tokens to enhance multilingual support. To improve the model's comprehension of numerical information, we decompose numbers into individual digits. Additionally, our tokenization strategy incorporates unicode-byte encoding as a fallback mechanism for rare characters, ensuring robust fault tolerance in text processing.

Enhanced Mathematical and Programming Content We have increased the proportion of mathematical and programming content in our pre-training corpus. Mathematical content is collected from Common Crawl using an iterative classification approach [Shao et al., 2024], supplemented with mathematical materials from books and academic papers. For programming content, we primarily utilize GitHub repositories, following cleaning procedures similar to [Guo et al., 2024]. To prevent data contamination in subsequent evaluations, we filter out entries sharing any 30-gram with the training or test sets of popular benchmarks, such as MATH [Hendrycks et al., 2021], GSM8K [Cobbe et al., 2021], HumanEval [Chen et al., 2021], and MBPP [Austin et al., 2021].

Semantic-based Document Organization Inspired by Shi et al. [2024], we implement large-scale clustering of documents with similar semantic features and concatenate them into extended sequences. These sequences are segmented into fixed-length pieces (8,192 tokens) for pre-training, with a high-quality subset reserved for subsequent long-context extension training.

Fine-grained Content Classification We develop a series of fine-grained classifiers for text types and topics, trained on annotations generated by smaller Yi models. The final pre-training data composition was determined through extensive experimentation with various dataset weighting schemes. We observed that focused training on a smaller volume of high-quality domain-specific data can enhance key model capabilities.

3.2 Training Strategy

Our training methodology follows a three-stage approach that optimizes learning rate schedules and strategic data sampling to maximize model performance [Ibrahim et al., 2024, Hu et al., 2024]: initial pre-training, mid-training, and fast-decay training.

The **initial pre-training** stage employs a warm-up schedule where the learning rate decays to half of its peak value. This strategy enables thorough exploration of the parameter space while avoiding premature convergence. During this stage, we emphasize data diversity to establish robust foundational capabilities across diverse domains.

In the **mid-training** stage, we focus on enhancing model capabilities and extending context length through gradual data distribution shifts. We implement an incremental upsampling strategy for high-quality data, emphasizing complex reasoning and multilingual capabilities for low-resource languages. We optimize training efficiency and improve throughput through dynamic batch size adjustments based on loss values.

The final **fast-decay training** stage, consuming about 12.5% of total training tokens, combines an aggressive learning rate decay with dynamic batch size optimization. This stage intensifies high-quality data upsampling and incorporates early instruction-tuning adaptation. It is notably designed to be iteratively flexible, allowing multiple optimization cycles tailored to specific deployment requirements for better practical utility.

3.3 Long Context Extension

After the fast-decay training stage, we apply additional long-context training to extend the context window to 64K tokens. We employ Rotary Position Embedding (RoPE) [Su et al., 2021] with

increased base frequency during extension [Xiong et al., 2023]. This training process systematically upsamples sequences from multiple length intervals (8K-16K, 16K-32K, and 32K-64K tokens) while maintaining consistent data distribution [Fu et al., 2024]. We found that a training corpus of 20B tokens successfully developed robust long-context capabilities without compromising performance on standard benchmarks.

4 Post-training

We next detail Yi-Lightning's post-training methodology. Our approach incorporates the sequential stages of Supervised Fine-Tuning (SFT) and Reinforcement Learning from Human Feedback (RLHF). We particularly elaborate on our strategies for multi-stage model training as well as data curation and synthesis.

4.1 Supervised Fine-tuning

4.1.1 Data and Training Strategy

Multi-stage Training and Data Curation Our supervised fine-tuning (SFT) process involves two sequential stages, leveraging 1.3M and 300,000 samples respectively. The first stage focuses on enhancing fundamental capabilities in mathematics and coding through extensive synthetic data, while the second stage utilizes diverse, high-quality general-domain data to boost instruction following and problem-solving capabilities. Furthermore, we implement a *small-to-large data scaling* strategy to expand our dataset systematically across both stages. We first compile a comprehensive prompt set from diverse sources using efficient selection strategies, and then generate corresponding responses through manual curation and synthetic approaches. For example, in the second stage, we methodically expanded from approximately 10,000 initial, high-quality seed samples to the target of 300,000 samples. This two-stage methodology, combined with the progressive data expansion strategy, effectively addresses data imbalance while rapidly enhancing model capabilities.

Synthetic Data Generation Synthetic data has proved instrumental, particularly for complex tasks including instruction following, code generation, and mathematical problem-solving. We employ multiple synthesis techniques including document augmentation, self-evolution, and language translation for prompt generation. For general tasks, we leverage multiple advanced models for response generation, combining automated systems and manual verification for quality control. For complex tasks like coding and mathematics, we integrate search algorithms, including Monte Carlo Tree Search (MCTS) and Depth-First Search (DFS), with specialized outcome and process reward models [Lightman et al., 2023] to generate diverse, accurate solutions. These methodologies yield a substantial corpus of high-quality, diverse training data, contributing significantly to our model's initial capabilities.

4.1.2 Optimized Implementations

We implement **sample packing**, which concatenates multiple samples into single sequences rather than applying individual padding. While this approach substantially reduces training sequences and improves efficiency, it can create artificial multi-turn contexts that potentially compromise certain model capabilities, particularly in multi-turn dialogues. We address this challenge by implementing **block causal attention (BCA)**, which isolates samples within sequences through masking matrices. We also observed that sample packing could introduce potential biases where longer samples disproportionately influence the total loss, potentially undermining short-sample task performance. We thus develop a **sample reweighting** mechanism that equalizes loss weights across all samples within a batch, which effectively mitigates the length-induced optimization bias.

4.2 Reinforcement Learning from Human Feedback

Training language models with human feedback has emerged as a crucial step for practical applications to align model behavior with human preferences [Bai et al., 2022, Ouyang et al., 2022, Zheng et al., 2023a, Rafailov et al., 2023, Xiong et al., 2024, Zheng et al., 2024a], which is also key to Yi-Lightning's exceptional performance on Chatbot Arena, where evaluations are based on direct human

comparisons and judgments. We next present a detailed discussion of our methodology for reward modeling (§ 4.2.1), preference data construction (§ 4.2.2), and alignment training (§ 4.2.3) procedures.

4.2.1 Reward Modeling

Following Bai et al. [2022, 2023], we implement a two-stage approach for reward model training: **preference model pre-training (PMP)** and **human-feedback fine-tuning (HFFT)**.

PMP Data Construction Our PMP dataset incorporates diverse preference datasets from public sources [Cui et al., 2023, Wang et al., 2024b, LLaMA Factory, 2024]. Given the varying quality standards and potential redundancies in these datasets, we implement rigorous cleaning and preprocessing protocols. We assess dataset quality by training individual reward models and evaluating their performance on our in-house benchmarks, retaining only those datasets yielding high benchmark performance.

HFFT Data Construction The HFFT dataset is constructed using comprehensive human annotations. We collect prompts from curated public datasets and then generate responses using model checkpoints from various SFT training phases. These responses are evaluated across category-specific dimensions. For instance, responses for coding prompts are assessed on instruction adherence, correctness, and code style. We form preference pairs by selecting the highest and lowest-scoring responses for each prompt, based on weighted dimensional scores, where the pairs with insufficient score differentials are excluded.

Reward Model Training With the above curated datasets, the reward model training initializes from a pre-trained model and proceeds through the two sequential stages (PMP and HFFT) using the Bradley-Terry loss [Bradley and Terry, 1952].

4.2.2 Preference Data

Prompts We generate prompts for preference learning through two approaches: collecting from public datasets and prompt synthesis. We first collect diverse prompts from various domains (coding, mathematics, general QA, etc.) to establish a comprehensive foundation across multiple instruction contexts. To enhance the model's capability in handling complex queries, we synthesize additional challenging prompts. We assign complexity scores based on instruction context complexity, analytical requirements, and output format specifications. High-scoring prompts are selected as *seed prompts* and paired with diverse *seed contexts* collected from high-quality web sources to create synthesized prompts. Finally, we apply multiple deduplication techniques, including n-gram similarity analysis, embedding-based comparisons, and random downsampling, to ensure the uniqueness of prompts while preserving their diversity.

Preference Pairs To ensure balanced data distribution and rational reward assignment, we categorize prompts across multiple dimensions: complexity level, user intent clarity, and domains (e.g., math, code, general QA). This categorization guides prompt balance adjustments and informs rating criteria weights for different categories. For each prompt, we generate multiple responses using the SFT model with varying temperature settings. These responses are evaluated using the reward model in § 4.2.1, and preference pairs are formed by selecting the highest and lowest-scoring responses while ensuring a sufficient reward gap to minimize the impact of reward modeling error.

4.2.3 Direct Preference Optimization

We conduct training for human preference alignment via the direct preference optimization (DPO) algorithm [Rafailov et al., 2023]. Inspired by recent work on iterative DPO training [Yang et al., 2024, Xiong et al., 2024], we conduct the DPO training in two sequential stages: offline and online training. In the **offline DPO training** stage, we train the model on the preference dataset constructed in § 4.2.2. In the **online DPO training** stage, we further extend the offline training with the real-time dataset generated by the most recent model. For each prompt, we sample 16 candidate responses and form a preference pair using the reward model in § 4.2.1, which is then used for model training in the next iteration. We conducted two iterations of online DPO training in total.

We enhance the training efficiency of the conventional DPO implementation through two key optimizations. First, instead of keeping the reference model loaded in GPU memory during training, we pre-compute and cache the log probability of samples from the preference dataset before each training iteration. These numerical values can then be fast indexed during training without the extra overhead of loading the reference model. Second, we leverage the fact that preference pairs share the same context to optimize computation. For each batch of preference pairs, we first process all positive samples followed by negative samples, reusing the KV-cache of their shared contexts. This optimization is particularly effective for long-context samples as it eliminates redundant context processing. Consequently, the two improvements substantially reduce GPU memory usage and enhance overall training efficiency.

5 Infrastructure

5.1 Parallelism Optimization

Given the architectural characteristics of MoE models (§ 2), we implement a hybrid parallelization strategy combining **expert parallelism** and **pipeline parallelism**. We further enhance the pipeline parallelism through several optimizations, including customized pipeline stage partitioning and fine-grained gradient recomputation strategies. These improvements enable optimal memory utilization and workload distribution across devices while maintaining training stability and enhancing overall throughput.

To fully exploit the advantages of both hybrid attention (§ 2.3) and context parallelism in long-context scenarios (§ 3.3), we introduce several refinements to the context parallelism implementation. These modifications enable efficient integration with the hybrid attention mechanism, particularly in optimizing the distribution of sliding window attention computations across the context parallel dimension. Our approach significantly reduces the computational burden on individual context parallel ranks, resulting in an up to 70% training speedup.

5.2 Inference Optimization

Yi-Lightning leverages a high-performance inference engine optimized specifically for LLM inference, effectively addressing the computational and memory bottlenecks. Through integrated algorithmic and engineering optimizations, the system achieves substantial reductions in resource consumption while delivering exceptional inference efficiency. The key optimizations include:

Advanced Asynchronous Scheduling at Engine Level Traditional LLM inference solutions often suffer from suboptimal GPU utilization (typically below 70%) due to serial dependencies between modules causing GPU idle time. We implement sophisticated multi-module, multi-process asynchronous scheduling that decouples task execution and minimizes inter-module latency. This enhancement achieves 95% GPU utilization in high-concurrency scenarios, markedly improving both engine performance and hardware resource efficiency.

Optimized FP8 Quantization and Hardware-Aware Operator Design Yi-Lightning's architecture is fundamentally designed with GPU hardware characteristics in mind, particularly for FP8 quantization compatibility. The model architecture precisely aligns with hardware specifications, maintaining algorithmic precision while maximizing hardware utilization. Our training infrastructure fully exploits the Nvidia Hopper architecture through custom-developed high-performance operators. A notable example is our implementation of the Mixture-of-Experts (MoE) operator, which employs an expert-parallel strategy achieving 1,200 TFLOPS per card at FP8 precision on Hopper GPUs. This represents a performance improvement exceeding 100% for operator execution, substantially enhancing overall inference efficiency.

The combined impact of these optimizations - enhanced hardware utilization through asynchronous scheduling and efficient operator implementation - enables Yi-Lightning to effectively address computational and memory constraints in high-concurrency, high-throughput inference scenarios, making it ideally suited for large-scale AI service deployment.

5.3 Goodput Optimization

Industry leaders like Meta [Dubey et al., 2024], Google [Gemini Team et al., 2023], and Alibaba [Dong et al., 2024] have achieved goodput levels exceeding 90% by employing advanced techniques such as fast checkpointing, fault-tolerant scheduling, and hardware redundancy. For instance, Meta's Grand Teton AI system [Bjorlin, 2022] leverages RDMA over RoCE and Infiniband networks to maintain high performance, while ByteDance's MegaScale [Jiang et al., 2024b] system focuses on rapid error detection and recovery to minimize downtime. Similarly, Google and Alibaba have implemented proactive hardware health monitoring and network optimizations to sustain near-maximum goodput even in the face of frequent hardware failures.

Building on these insights, we adopt a multi-layered approach to goodput optimization in our large-scale GPU cluster, **XCloud**:

Fault Tolerance through Proactive and Reactive Mechanisms One of the most significant contributions to goodput optimization is the combination of proactive and reactive fault discovery strategies. Proactive measures such as routine, entrance, and preflight tests ensure cluster health by identifying potential hardware and software issues before they impact workloads. On the reactive side, XCloud employs advanced monitoring tools like node exporters[1] and custom InfiniBand metrics collectors to detect faults in real time. These systems work in tandem to minimize the duration and impact of failures, enabling rapid recovery and reducing wasted computational resources. This dual-layer approach ensures that GPU resources remain optimally utilized even in the face of frequent hardware or network failures.

Memory-Based Asynchronous Checkpointing Traditional checkpointing systems, like those relying on distributed file systems (e.g., GPFS [Schmuck and Haskin, 2002]), often introduce significant overhead, leading to idle GPU time during save operations. XCloud's memory-based asynchronous checkpointing drastically reduces the time required to save model states, from several minutes to just 3–5 seconds. This innovation not only minimizes the GPU idle period but also encourages more frequent checkpointing, reducing computational waste during recovery. The result is a substantial boost in system resilience and overall efficiency, contributing directly to achieving and maintaining goodput levels above 99%.

6 Safety

As large language models continue to grow in capability, it is crucial to ensure their safe and responsible operation across varying and complex scenarios [Achiam et al., 2023, Dubey et al., 2024, Zou et al., 2023, Wei et al., 2023, Zheng et al., 2024b, Wang et al., 2024a, Zhao et al., 2024]. To address the safety concerns of Yi-Lightning, we develop **RAISE (Responsible AI Safety Engine)**, a comprehensive safety framework illustrated in Figure 2. RAISE is designed to provide robust safety capabilities throughout the model's entire lifecycle, from development to deployment, effectively minimizing potential risks and threats through systematic safety mechanisms.

The RAISE framework comprises four integral components (RAISE-1 to RAISE-4), corresponding to pre-training, post-training, and inference-time input/output processing. Through sophisticated technical approaches and their synergistic integration, these components collectively ensure model safety while maintaining optimal user experience.

RAISE-1: Pre-training Safety At the pre-training phase, we implement a safety model for pre-training data filtration. We develop classification models based on Transformer and DNN architectures, trained on high-quality compliant datasets. These models form an evaluation and filtering pipeline for the pre-training corpus, ensuring data reliability, minimizing erroneous information and biased content, preventing privacy data leakage, and enhancing model safety and compliance.

RAISE-2: Post-training Optimization During post-training, we implement fine-tuning strategies to optimize safety performance across different application scenarios. Our approach incorporates evaluation and scoring mechanisms during SFT and RLHF stages, using reward engineering to

[1] `https://github.com/prometheus/node_exporter`

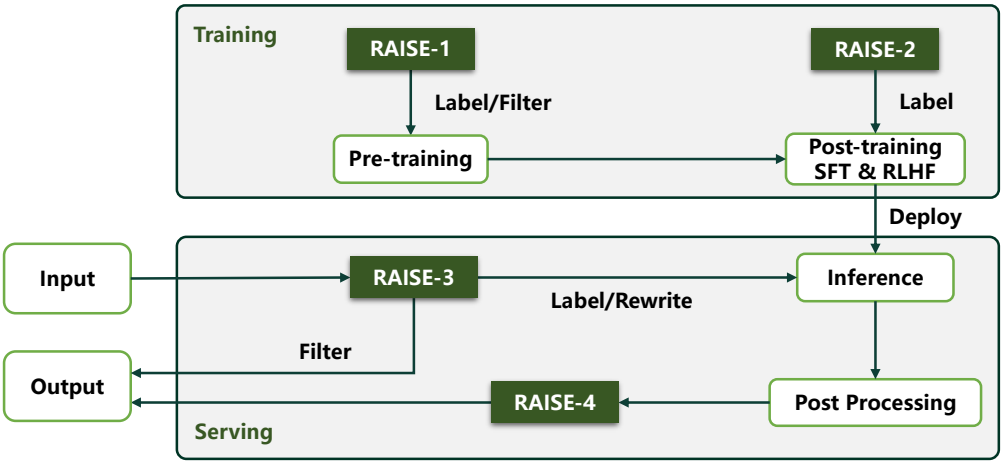

Figure 2: Illustration of RAISE (Responsible AI Safety Engine) system.

encourage safe responses and penalize potentially harmful outputs. The additional quality control processes further ensure proper value alignment while maintaining core model performance.

RAISE-3: Input Safety Processing For inference-time input processing, we deploy safety assessment mechanisms that analyze and filter content. The system identifies potentially harmful content, including malicious, discriminatory, or hateful elements, while ensuring input safety and compliance. These mechanisms minimize risks of the model being manipulated while maintaining performance under various input conditions.

RAISE-4: Output Safety Control The output safety control system implements real-time detection and optimization across key dimensions: value alignment, bias detection, legal compliance, accuracy assessment, and content appropriateness. This component integrates safety mechanisms to ensure output quality while maintaining efficiency, balancing safety requirements with response speed.

Through this framework, RAISE provides a foundation for responsible AI development and deployment, ensuring safety across Yi-Lightning's lifecycle while maintaining performance and user satisfaction. The interaction between these components creates a safety ecosystem addressing both current and emerging challenges.

7 Evaluation

Chatbot Arena As shown in Figure 1, with the initial appearance on Chatbot Arena[2] [Zheng et al., 2023b] on October 16, 2024, our flagship model Yi-Lightning achieved a remarkable overall ranking of **6th** place (Arena score **1287**), performing on par with GPT-4o-0513 (ranked 7th, Arena score 1285). In specialized rankings, Yi-Lightning also exhibited remarkable performance: **2nd** in *Chinese*, **3rd** in *Multi-Turn* and *Math*, and **4th** in *Coding*, *Hard Prompts*, and *Longer Query* categories. Since the Chatbot Arena rankings derive from authentic human comparisons and voting, these results powerfully demonstrate Yi-Lightning's exceptional ability to fulfill user needs and its satisfactory alignment with human preferences in real-world applications.

Academic Benchmarks We report the evaluation results on several representative, public academic benchmarks: GPQA [Rein et al., 2023] for general knowledge, MATH [Hendrycks et al., 2021] for mathematical reasoning, HumanEval [Chen et al., 2021] for coding, and IFEval [Zhou et al., 2023] for instruction following. We also conduct the LLM-as-a-judge evaluations[3] on WildBench [Lin et al.,

[2] `https://lmarena.ai/`

[3] We employ *GPT-4o-0513* for all the LLM-as-a-judge evaluations.

2024], Arena-Hard[4] [Li et al., 2024], AlignBench-v1.1 [Liu et al., 2023], and MT-Bench [Zheng et al., 2023b].

We present in Table 1 the comparison between Yi-Lightning and the top-tier open-weight LLMs, including Qwen2.5-72B-Instruct [Team, 2024], DeepSeek-V2.5 [Liu et al., 2024], Mistral-Large-Instruct-2407 [Jiang et al., 2023], and Llama3.1-70B/405B-Instruct [Dubey et al., 2024]. We also show in Table 2 the comparison with the top-tier proprietary LLMs, including GPT-4o-0513, Claude-3.5-Sonnet-20240620, and our last-generation model Yi-Large-Preview. Overall, Yi-Lightning remains competitive on these academic benchmarks.

Table 1: Comparison with the top-tier open-weight LLMs on public academic benchmarks.

	Qwen2.5 72B Instruct	DeepSeek V2.5	Mistral Large Instruct-2407	Llama3.1 70B Instruct	Llama3.1 405B Instruct-FP8	**Yi-Lightning**
GPQA *0-shot*	49.1	42.4	50.1	45.1	**53.0**	50.9
MATH *0-shot*	**82.7**	73.9	73.3	67.1	67.7	76.4
HumanEval *0-shot*	**86.0**	85.4	**86.0**	76.2	84.1	83.5
IFEval *Prompt Loose*	86.0	82.1	82.8	87.1	**88.5**	81.9
WildBench *Judge: GPT-4o-0513*	59.9	57.9	57.4	49.0	51.6	**65.1**
Arena-Hard *Judge: GPT-4o-0513*	90.5	88.3	85.1	74.0	71.2	**91.8**
AlignBench-v1.1 *Judge: GPT-4o-0513*	7.51	7.38	7.10	5.81	5.56	**7.54**
MT-Bench *Judge: GPT-4o-0513*	8.62	8.43	8.53	8.23	8.36	**8.75**

Table 2: Comparison with the top-tier proprietary LLMs (GPT-4o-0513 and Claude-3.5-Sonnet-20240620) and our previous-generation model (Yi-Large-Preview) on public academic benchmarks.

	Yi-Large Preview	GPT-4o 0513	Claude-3.5-Sonnet 20240620	**Yi-Lightning**
GPQA *0-shot*	43.8	51.9	**57.8**	50.9
MATH *0-shot*	62.6	76.0	74.0	**76.4**
HumanEval *0-shot*	75.6	**90.5**	88.3	83.5
IFEval *Prompt Loose*	79.3	87.6	**88.5**	81.9
WildBench *Judge: GPT-4o-0513*	55.3	59.3	54.7	**65.1**
Arena-Hard *Judge: GPT-4o-0513*	79.1	**92.9**	85.6	91.8
AlignBench-v1.1 *Judge: GPT-4o-0513*	7.20	**7.59**	7.17	7.54
MT-Bench *Judge: GPT-4o-0513*	8.32	8.59	6.96	**8.75**

[4]Since Yi-Lightning's API serving trades off inference speed and accuracy, evaluation results based on the API serving may be slightly lower than our reported evaluation results based on local deployment.

Final Discussion

Finally, we discuss our observed disparity between open-weight and proprietary models' performance on public academic benchmarks and real-world user preferences (as reflected in the Chatbot Arena rankings). This probably results from the fact that our development process paid more attention to our in-house human assessment experience, rather than overly focusing on academic benchmark scores. For instance, when conducting math-specific model training (e.g., in § 4.1), we did not strictly restrict the model's output format (e.g., ending with "The final answer is \boxed{...}"), as we believe that constraining the model's output content or format might harm its generation diversity, thereby implicitly impacting optimization effectiveness and user experience. These evaluation results prompt us to rethink and reassess the role of public academic benchmarks in guiding the development of more intelligent and powerful AI systems.

A Author List and Contributions

We list our team members in alphabetical order. All authors contribute equally to this work.

Pre-training

- Bei Chen
- Chao Li
- Chengen Huang
- Fan Zhou
- Ge Zhang
- Jun Tian
- Peng Liu
- Shiming Yang
- Wenhao Huang
- Xiaoyi Ren
- Xinyao Niu
- Yanpeng Li
- Yuchi Xu
- Zhiyuan Liu

Post-training

- C.X. Lv
- Chenglin Cai
- Chujie Zheng
- Guoyin Wang
- Heng Ji
- Katherine Su
- Ming Song
- Shawn Wang
- Tianhang Zhu
- Xiang He
- Xiaohui Hu
- Yuxuan Sha

Infrastructure

- Feng Hu
- Jiangcheng Zhu
- Lihuan Zhang
- Liying Li
- Mou Li
- Qicheng Hu
- Wen Xie
- Xiaobo Chen
- Yongke Zhao
- Zhaodong Yan
- Zirui Zhang
- Zonghong Dai

Safety

- Shijun Zhou
- Shiyong Li
- Yongzhen Luo

Evaluation

- Alan Wake
- Daniel Cooper
- Howard Qiu

References

01.AI. Yi: Open foundation models by 01.ai. *arXiv preprint arXiv:2403.04652*, 2024.

Marah Abdin, Jyoti Aneja, Hany Awadalla, Ahmed Awadallah, Ammar Ahmad Awan, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Jianmin Bao, Harkirat Behl, et al. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*, 2024.

Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.

Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022.

Alexis Bjorlin. Ocp summit 2022: Open hardware for ai infrastructure, Nov 2022. URL `https://engineering.fb.com/2022/10/18/open-source/ocp-summit-2022-grand-teton/`.

Ralph Allan Bradley and Milton E Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Ganqu Cui, Lifan Yuan, Ning Ding, Guanming Yao, Wei Zhu, Yuan Ni, Guotong Xie, Zhiyuan Liu, and Maosong Sun. Ultrafeedback: Boosting language models with high-quality feedback. *arXiv preprint arXiv:2310.01377*, 2023.

Damai Dai, Chengqi Deng, Chenggang Zhao, RX Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y Wu, et al. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066*, 2024.

Jianbo Dong, Bin Luo, Jun Zhang, Pengcheng Zhang, Fei Feng, Yikai Zhu, Ang Liu, Zian Chen, Yi Shi, Hairong Jiao, et al. Boosting large-scale parallel training efficiency with c4: A communication-driven approach. *arXiv preprint arXiv:2406.04594*, 2024.

Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

Yao Fu, Rameswar Panda, Xinyao Niu, Xiang Yue, Hannaneh Hajishirzi, Yoon Kim, and Hao Peng. Data engineering for scaling language models to 128k context. *arXiv preprint arXiv:2402.10171*, 2024.

Gemini Gemini Team, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, et al. Gemini: A family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.

Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. Deepseek-coder: When the large language model meets programming – the rise of code intelligence. *arXiv preprint arXiv:2401.14196*, 2024.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.

Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu Cui, Xiang Long, Zhi Zheng, Yewei Fang, Yuxiang Huang, Weilin Zhao, et al. Minicpm: Unveiling the potential of small language models with scalable training strategies. *arXiv preprint arXiv:2404.06395*, 2024.

Adam Ibrahim, Benjamin Thérien, Kshitij Gupta, Mats L Richter, Quentin Anthony, Timothée Lesort, Eugene Belilovsky, and Irina Rish. Simple and scalable strategies to continually pre-train large language models. *arXiv preprint arXiv:2403.08763*, 2024.

Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.

Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024a.

Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, et al. {MegaScale}: Scaling large language model training to more than 10,000 {GPUs}. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 745–760, 2024b.

Taku Kudo and John Richardson. SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing. *arXiv preprint arXiv:1808.06226*, 2018.

Tianle Li, Wei-Lin Chiang, Evan Frick, Lisa Dunlap, Tianhao Wu, Banghua Zhu, Joseph E Gonzalez, and Ion Stoica. From crowdsourced data to high-quality benchmarks: Arena-hard and benchbuilder pipeline. *arXiv preprint arXiv:2406.11939*, 2024.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.

Bill Yuchen Lin, Yuntian Deng, Khyathi Chandu, Faeze Brahman, Abhilasha Ravichander, Valentina Pyatkin, Nouha Dziri, Ronan Le Bras, and Yejin Choi. Wildbench: Benchmarking llms with challenging tasks from real users in the wild. *arXiv preprint arXiv:2406.04770*, 2024.

Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Dengr, Chong Ruan, Damai Dai, Daya Guo, et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

Xiao Liu, Xuanyu Lei, Shengyuan Wang, Yue Huang, Zhuoer Feng, Bosi Wen, Jiale Cheng, Pei Ke, Yifan Xu, Weng Lam Tam, et al. Alignbench: Benchmarking chinese alignment of large language models. *arXiv preprint arXiv:2311.18743*, 2023.

LLaMA Factory. Dpo-en-zh-20k (revision 21ff425), 2024. URL `https://huggingface.co/datasets/llamafactory/DPO-En-Zh-20k`.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training Language Models to Follow Instructions with Human Feedback. *Advances in Neural Information Processing Systems*, 35: 27730–27744, 2022.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *arXiv preprint arXiv:2305.18290*, 2023.

David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R Bowman. Gpqa: A graduate-level google-proof q&a benchmark. *arXiv preprint arXiv:2311.12022*, 2023.

Frank Schmuck and Roger Haskin. GPFS: A Shared-Disk file system for large computing clusters. In *Conference on File and Storage Technologies (FAST 02)*, Monterey, CA, January 2002. USENIX Association. URL `https://www.usenix.org/conference/fast-02/gpfs-shared-disk-file-system-large-computing-clusters`.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Weijia Shi, Sewon Min, Maria Lomeli, Chunting Zhou, Margaret Li, Gergely Szilvasy, Rich James, Xi Victoria Lin, Noah A. Smith, Luke Zettlemoyer, Scott Yih, and Mike Lewis. In-context pretraining: Language modeling beyond document boundaries. *arXiv preprint arXiv:2310.10638*, 2024.

Yusuxke Shibata, Takuya Kida, Shuichi Fukamachi, Masayuki Takeda, Ayumi Shinohara, Takeshi Shinohara, and Setsuo Arikawa. Byte Pair Encoding: A Text Compression Scheme That Accelerates Pattern Matching. Technical report, Technical Report DOI-TR-161, Department of Informatics, Kyushu University, 1999.

Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced Transformer with Rotary Position Embedding. *arXiv preprint arXiv:2104.09864*, 2021.

Qwen Team. Qwen2.5: A party of foundation models, September 2024. URL `https://qwenlm.github.io/blog/qwen2.5/`.

Shenzhi Wang, Chang Liu, Zilong Zheng, Siyuan Qi, Shuo Chen, Qisen Yang, Andrew Zhao, Chaofei Wang, Shiji Song, and Gao Huang. Boosting LLM agents with recursive contemplation for effective deception handling. In *Findings of the Association for Computational Linguistics: ACL*, 2024a.

Zhilin Wang, Yi Dong, Olivier Delalleau, Jiaqi Zeng, Gerald Shen, Daniel Egert, Jimmy J Zhang, Makesh Narsimhan Sreedhar, and Oleksii Kuchaiev. Helpsteer2: Open-source dataset for training top-performing reward models. *arXiv preprint arXiv:2406.08673*, 2024b.

Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. Jailbroken: How does llm safety training fail? In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Wei Xiong, Hanze Dong, Chenlu Ye, Ziqi Wang, Han Zhong, Heng Ji, Nan Jiang, and Tong Zhang. Iterative preference learning from human feedback: Bridging theory and practice for RLHF under KL-constraint. In *ICLR 2024 Workshop on Mathematical and Empirical Understanding of Foundation Models*, 2024. URL `https://openreview.net/forum?id=w3oJXMAQx2`.

Wenhan Xiong, Jingyu Liu, Igor Molybog, Hejia Zhang, Prajjwal Bhargava, Rui Hou, Louis Martin, Rashi Rungta, Karthik Abinav Sankararaman, Barlas Oguz, et al. Effective long-context scaling of foundation models. *arXiv preprint arXiv:2309.16039*, 2023.

An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024.

Zhengyan Zhang, Yixin Song, Guanghui Yu, Xu Han, Yankai Lin, Chaojun Xiao, Chenyang Song, Zhiyuan Liu, Zeyu Mi, and Maosong Sun. Relu^2 wins: Discovering efficient activation functions for sparse llms. *arXiv preprint arXiv:2402.03804*, 2024a.

Zhengyan Zhang, Chaojun Xiao, Qiujieli Qin, Yankai Lin, Zhiyuan Zeng, Xu Han, Zhiyuan Liu, Ruobing Xie, Maosong Sun, and Jie Zhou. Exploring the benefit of activation sparsity in pre-training. *arXiv preprint arXiv:2410.03440*, 2024b.

Andrew Zhao, Quentin Xu, Matthieu Lin, Shenzhi Wang, Yong-jin Liu, Zilong Zheng, and Gao Huang. Diver-ct: Diversity-enhanced red teaming with relaxing constraints. *arXiv preprint arXiv:2405.19026*, 2024.

Chujie Zheng, Pei Ke, Zheng Zhang, and Minlie Huang. Click: Controllable text generation with sequence likelihood contrastive learning. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 1022–1040, Toronto, Canada, July 2023a. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-acl.65. URL `https://aclanthology.org/2023.findings-acl.65`.

Chujie Zheng, Ziqi Wang, Heng Ji, Minlie Huang, and Nanyun Peng. Weak-to-strong extrapolation expedites alignment. *arXiv preprint arXiv:2404.16792*, 2024a.

Chujie Zheng, Fan Yin, Hao Zhou, Fandong Meng, Jie Zhou, Kai-Wei Chang, Minlie Huang, and Nanyun Peng. On prompt-driven safeguarding for large language models. In *International Conference on Machine Learning*, 2024b.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Advances in Neural Information Processing Systems*, volume 36, pages 46595–46623, 2023b.

Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023.

Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.